Clase 08: Multiprocessing en Python

IIC3533 Computación de Alto Rendimiento

Ignacio Labarca – ignacio.labarca@uc.cl

Departamento de Ciencia de la Computación
Escuela de Ingeniería
Pontificia Universidad Católica de Chile

Martes 31 de Marzo 2026

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

Hilos (*threads*)

Un **hilo** es la unidad mínima de ejecución dentro de un proceso.

- Varios hilos comparten el mismo espacio de memoria del proceso.
- El sistema operativo los planifica de forma independiente.
- Permiten ejecutar varias tareas *aparentemente* en paralelo dentro de un mismo programa.
- Cada *core* puede ejecutar un hilo a la vez: con N cores se pueden ejecutar N hilos simultáneamente.
- Si hay más hilos que cores, el SO los intercala.

Procesos

Un **proceso** es un programa en ejecución, con su propio espacio de memoria.

- Al crearse, el SO le asigna su propio espacio de memoria: stack y heap.
- Cada proceso tiene su propia memoria: no comparte datos con otros procesos.
- Un proceso puede tener múltiples hilos, todos compartiendo su memoria.
- La comunicación entre procesos requiere mecanismos explícitos.

El intérprete de Python

CPython ejecuta el código línea por línea: lo traduce a *bytecode* y lo interpreta en su máquina virtual.

- El intérprete mantiene un estado global (conteo de referencias, memoria).
- Dos hilos modificando ese estado simultáneamente pueden corromperse.
- Ante eso, CPython usa el **Global Interpreter Lock (GIL)**.

El GIL

El **Global Interpreter Lock (GIL)** permite que solo un hilo ejecute bytecode Python a la vez, incluso en sistemas multicore.

- Los *threads* de Python son útiles para I/O, pero no para cómputo.
- Código CPU-bound con múltiples *threads* puede ser más lento que secuencial.
- Ante eso, la alternativa es usar procesos separados, cada uno con su propio intérprete y GIL.

El módulo `threading`

Python tiene el módulo `threading` para crear y manejar hilos.

- Útil para tareas I/O-bound: lectura de archivos, requests de red, etc.
- Para tareas CPU-bound, el GIL impide el paralelismo real.
- En la práctica, para cómputo en Python se prefiere `multiprocessing`.

El módulo `multiprocessing`

Lanzar procesos separados, cada uno con su propio intérprete Python y su propio GIL.

Ventajas:

- Paralelismo real en sistemas multicore.
- Cada proceso es independiente: fallos aislados.
- API similar a `threading`.

Costos:

- Mayor *overhead* de creación que los hilos.
- Los datos deben ser serializados (*pickle*) para pasar entre procesos.

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

Creación de procesos

Al crear un proceso hijo, el SO puede hacerlo de dos formas:

fork (por defecto en Linux):

- Copia el proceso padre completo: el hijo hereda memoria, variables y recursos abiertos (conexiones, file descriptors).
- El estado heredado puede causar comportamiento inesperado.

spawn (por defecto en macOS y Windows):

- Crea un proceso Python limpio y reimporta el módulo principal.
- Más seguro, pero más lento debido al costo de setup.

Hello World con multiprocessing

```python
import multiprocessing as mp

def worker(pid):
    print(f"Hola desde proceso {pid}")

if __name__ == "__main__":
    processes = []
    for i in range(4):
        p = mp.Process(target=worker, args=(i,))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()
```

La clase Process

```
p = mp.Process(target=func, args=(arg1, arg2), kwargs={...})
p.start()     # lanza el proceso
p.join()      # espera a que termine
p.terminate() # termina el proceso
p.is_alive()  # True si esta corriendo
```

- Cada proceso tiene su propio espacio de memoria.
- Los datos se copian al proceso hijo (fork en Linux, spawn en Windows).
- Cambios en el proceso hijo no afectan al padre.

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

Pool: paralelismo sobre colecciones

El patrón más común: aplicar una función a muchos datos en paralelo.

```
import multiprocessing as mp

def cuadrado(x):
    return x * x

if __name__ == "__main__":
    datos = list(range(10))
    with mp.Pool(processes=4) as pool:
        resultados = pool.map(cuadrado, datos)
    print(resultados)
    # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Métodos de Pool

- `pool.map(f, iterable)`: aplica `f` a cada elemento; el padre espera hasta terminar.
- `pool.map_async(f, iterable)`: versión asíncrona; retorna un objeto `AsyncResult`.
- `pool.starmap(f, iterable)`: como `map`, pero permite múltiples argumentos.
- `pool.apply(f, args)`: llama `f` en un proceso del pool; el padre espera.
- `pool.apply_async(f, args)`: versión asíncrona de `apply`.

starmap: múltiples argumentos

```python
import multiprocessing as mp

def potencia(base, exp):
    return base ** exp

if __name__ == "__main__":
    pares = [(2, 10), (3, 5), (4, 3), (10, 2)]
    with mp.Pool() as pool:
        resultados = pool.starmap(potencia, pares)
    print(resultados)
    # [1024, 243, 64, 100]
```

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

¿Cómo comunicar procesos?

Como los procesos no comparten memoria, se necesitan mecanismos explícitos.

- **Queue**: cola FIFO segura entre procesos.
- **Pipe**: canal de comunicación bidireccional entre dos procesos.
- **Memoria compartida**: `Value`, `Array`, `shared_memory`.

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

Primitivas de sincronización

`multiprocessing` ofrece las mismas primitivas que `threading`:

- `Lock`: exclusión mutua; solo un proceso a la vez en la sección crítica.
- `RLock`: *reentrant lock*; el mismo proceso puede adquirirlo múltiples veces.
- `Semaphore`: permite hasta N procesos simultáneos en una sección.
- `Event`: señal entre procesos (`set()`, `wait()`, `clear()`).
- `Barrier`: punto de sincronización: todos los procesos esperan hasta que lleguen N.

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

`joblib`: paralelismo de alto nivel

`joblib` es una librería de alto nivel orientada a facilitar el paralelismo en Python.

- API más simple que `multiprocessing`.
- Soporta múltiples *backends*: `loky` (procesos, por defecto), `threading`, `multiprocessing`.
- Muy usada en librerías científicas (scikit-learn, etc.).
- Incluye caché de resultados (`Memory`).

Instalación: `pip install joblib`

Parallel y delayed

El patrón central de `joblib`:

```
from joblib import Parallel, delayed

def cuadrado(x):
    return x * x

resultados = Parallel(n_jobs=4)(map(delayed(cuadrado),
    range(10)))
print(resultados)
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- `n_jobs=-1` usa todos los cores disponibles.
- `delayed(f)(args)` empaqueta la llamada para ejecutarla en paralelo.

Backends de joblib

```python
from joblib import Parallel, delayed

# Backend por defecto: loky (procesos)
Parallel(n_jobs=4)(map(delayed(f), data))

# Backend threading
Parallel(n_jobs=4, backend="threading")(map(delayed(f), data))

# Backend multiprocessing
Parallel(n_jobs=4, backend="multiprocessing")(map(delayed(f),
    data))
```

- `loky`: procesos aislados, robusto, por defecto.
- `threading`: útil cuando la función llama a código compilado (NumPy, etc.) que libera el GIL.
- `multiprocessing`: backend clásico de Python.

Backends y paralelismo real

No todos los backends ofrecen paralelismo real:

- `loky` y `multiprocessing`: cada worker es un proceso separado con su propio intérprete y GIL. Hay paralelismo real.
- `threading`: los workers son hilos del mismo proceso. El GIL impide paralelismo real para código Python puro: solo hay alternancia entre hilos.

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

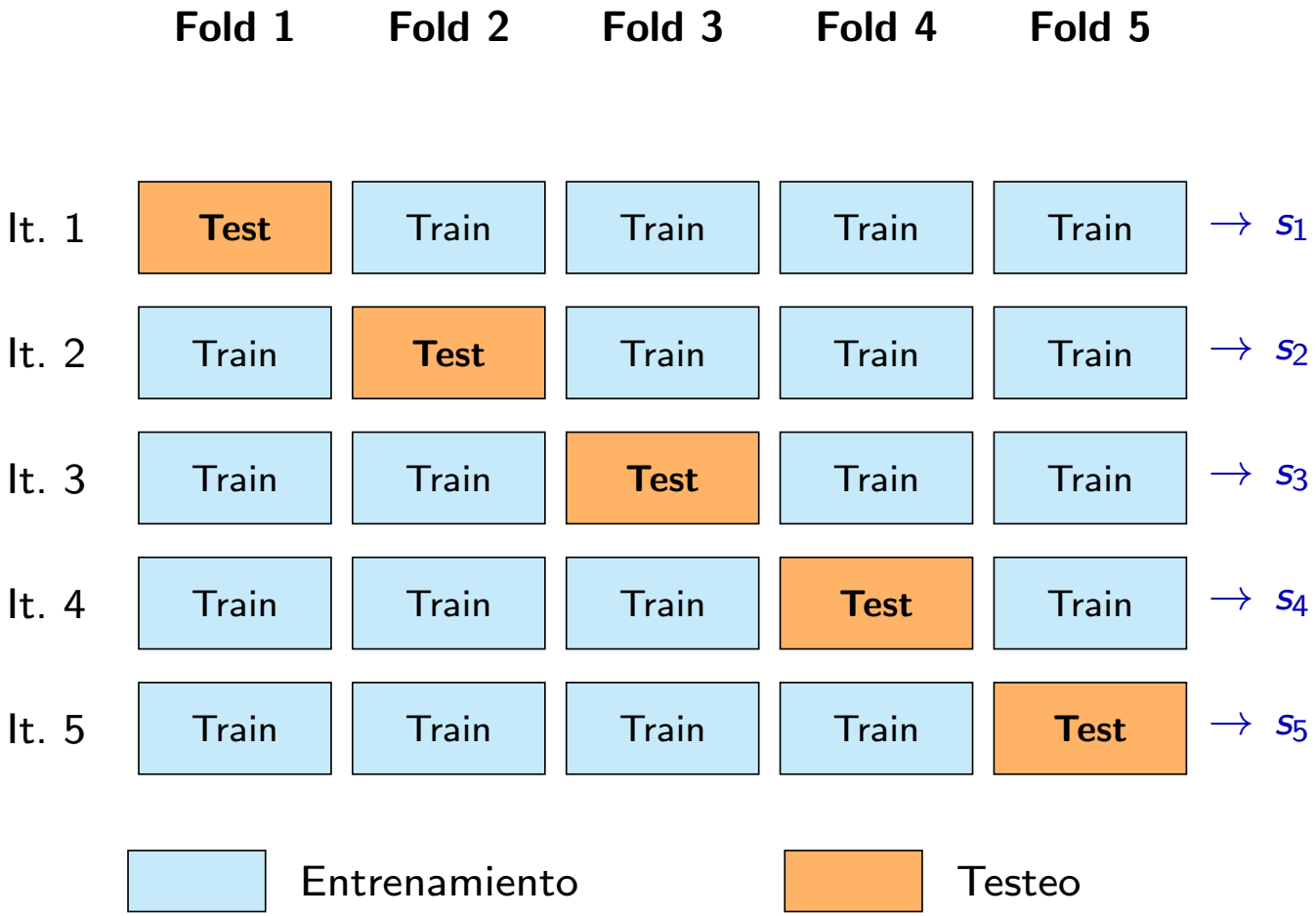
Validación cruzada (*k-fold cross-validation*)

Estrategia estándar para evaluar modelos de *machine learning*.

- Se divide el dataset en k *folds*.
- En cada iteración: se entrena con $k-1$ folds y se testea con el restante.
- Se repite k veces, rotando el fold de evaluación.
- El resultado final es el promedio de las k métricas.

Cada fold es completamente independiente y el entrenamiento es CPU-bound: ideal para multiprocessing.

5-fold cross-validation: diagrama



Resultado final: $\bar{s} = \frac{1}{5} \sum_{i=1}^{5} s_i$

Cross-validation secuencial

```python
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
import numpy as np

X, y = make_classification(n_samples=10000, n_features=20)
modelo = RandomForestClassifier(n_estimators=200)

scores = cross_val_score(modelo, X, y, cv=8)
print(f"Accuracy: {scores.mean():.4f} +/- {scores.std():.4f}")
```

Cross-validation paralelo con `joblib`

```
scores = cross_val_score(modelo, X, y, cv=8, n_jobs=-1)
print(f"Accuracy: {scores.mean():.4f} +/- {scores.std():.4f}")
```

`scikit-learn` delega el paralelismo a `joblib` internamente.

Consideraciones de rendimiento

- **Overhead de creación**: lanzar procesos tiene costo; `joblib` reutiliza procesos internamente.
- **Serialización**: los datos se serializan (*pickle*) para pasar entre procesos. Con datasets grandes, esto puede ser el cuello de botella.
- **Número de procesos**: idealmente igual a la cantidad de *cores* físicos (`n_jobs=-1`).
- **Granularidad**: tareas demasiado pequeñas no justifican el overhead entre procesos.

Motivación: el GIL de Python

El módulo multiprocessing

Pool de procesos

Comunicación entre procesos

Sincronización

joblib

Ejemplo: Validación cruzada

Ejemplo: Grid Search

Grid search manual

Evaluar todas las combinaciones de hiperparámetros de un modelo.

Cada combinación es independiente: ideal para multiprocessing.

Posibles modelos e hiperparámetros:

- **KNN**: `k`, métrica de distancia (`euclidean`, `manhattan`).
- **Árbol de decisión**: `max_depth`, `min_samples_split`, `criterion`.
- **Logistic Regression**: `C`, `penalty` (`l1`, `l2`).
- **K-Means**: número de clusters `k`.

Limitaciones

Los métodos vistos hoy están pensados para funcionar en un solo nodo.

- El limitante principal es la memoria RAM: todos los datos deben caber en un solo nodo.
- Para escalar a múltiples nodos se necesitan otras herramientas: **MPI**, **Dask**, **Ray**, **Spark**.